

SoK: Authorization for Autonomous Agents

Anonymous Author(s)

ABSTRACT

An autonomous agent calls tools and Model Context Protocol (MCP) servers while carrying a bearer token (an OAuth 2.0 access token or an MCP authorization token) on each request. Because the agent is driven by untrusted input, its authorization layer is part of the attack surface: a token not bound to the server it is sent to can be replayed or passed through, an over-broad scope hands an injected agent far more than any task needs, and an on-behalf-of token with no verifiable delegation chain leaves “who authorized this?” unanswerable. A fast-growing research literature studies agent security, but it concentrates on model-level attacks (prompt injection, tool poisoning) and proposes individual mechanisms; it does not systematize the authorization layer. We survey 44 research works and show that two authorization properties, audience binding and confused-deputy / dynamic client registration, are treated as mere specification obligations rather than as properties anyone tests or enforces. We then systematize agent authorization along three axes no prior work combines: (i) by *pre-deployment testability*, meaning which threats are decidable from a token, a key set, and a declared tool contract before the agent runs; (ii) by *normative provenance*, a threat-to-claim mapping to the OAuth RFCs, the MCP authorization specification, OWASP, and CWE; and (iii) by the *static-versus-runtime enforcement line*. Across fifteen threats in six categories, we find six are statically contract-verifiable and nine are irreducibly runtime, and we argue this line should guide both tooling and standardization.

1 INTRODUCTION

When an agent calls a tool or an MCP server on a user’s behalf, a bearer token travels with the request. The failure modes are OAuth failure modes, but with sharper consequences, because the caller is autonomous and can be steered by untrusted input such as a retrieved document or a tool description [?]. A token not bound to the resource it is presented to can be replayed against, or forwarded through, a different server (a confused deputy); an on-behalf-of (OBO) token with no verifiable delegation chain makes accountability impossible; an omnibus scope such as `admin:*` hands an injected agent the keys to everything; and a non-expiring credential never stops being useful to whoever obtains it.

A large and fast-moving body of research now studies agent and MCP security. As our survey (§5) shows, it is dense on *model-level* attacks (prompt-injection-driven escalation and tool poisoning) and it proposes many *individual* defense mechanisms (privilege controllers, delegation protocols, identity architectures). What it does not do is systematize the *authorization* layer itself: no surveyed work sorts agent-authorization threats by whether a defender can test them before deployment, maps each threat to the specific normative clause it violates, or names the line

between what can be enforced statically at the trust boundary and what can only be enforced at runtime. The two authorization properties that matter most for token safety, audience binding and the confused-deputy / dynamic-client-registration pair, are treated across 44 works as specification obligations to cite rather than as properties to verify (§5).

Contributions. (i) We give a source-grounded threat model of fifteen agent-authorization threats in six categories, each with its broken invariant, the conditions under which it is exploitable, a mitigation, and its normative source (§2, §3). (ii) We systematize this space along three axes no prior work combines: testability, normative provenance, and the static/runtime enforcement line (§4). (iii) We survey 44 research works on agent authorization and make the coverage gap precise: audience binding and confused-deputy are the under-formalized seam (§5). (iv) We argue which authorization properties a standard or a pre-deployment tool could mandate, and release an open-source analyzer for the statically decidable subset (§4).

2 BACKGROUND

Tokens and claims. An OAuth 2.0 access token authorizes a client to call a resource on a user’s behalf; in agent deployments it is usually a JSON Web Token (JWT), a signed set of claims. The claims that govern authorization are the issuer (iss), the intended audience (aud, the resource the token is *for*), the granted scope, the expiry (exp), and, for delegation, the authorized party (azp, naming the client) and the actor (act, naming who is acting). A verifier must check the signature against the issuer’s keys, that aud names itself, that the algorithm is on an allow-list, and that required claims hold (RFC 8725). RFC 9068 profiles JWT access tokens and makes audience and issuer validation mandatory for that profile; RFC 8707 (resource indicators) lets a client request a token bound to a specific resource.

Delegation and the act/azp claims. OAuth token exchange (RFC 8693) issues a delegated token carrying an act claim that names the acting party and a may_act claim stating who is *permitted* to act for the subject. A single act claim records one hop; a chain (user → agent → sub-agent) requires *nested* act claims, and transitive delegation is only verifiable if each exchange enforces may_act and the verifier walks the full nested chain. azp names the client that obtained the token but does not by itself establish a delegation chain: it identifies the immediate party, not the sequence of authorizations behind it. This is why azp/act alone are necessary but not sufficient: without enforced may_act and nested-chain validation, “who authorized this?” remains unanswerable across more than one hop.

MCP, identity, and the confused deputy. The Model Context Protocol standardizes how an agent discovers and calls tools; its authorization specification inherits OAuth and *prohibits* passing a client’s token through to an upstream API. Dynamic client registration (DCR, RFC 7591) lets clients register at runtime, which an MCP proxy may expose. Workload-identity substrates (WIMSE;

SPIFFE/SPIRE) would give each agent instance a verifiable, short-lived identity distinct from the human's, but are rarely deployed. The confused-deputy problem (CWE-441), where a more privileged party misuses its authority on behalf of a less privileged one, is the classical frame for token passthrough and consent-flow abuse.

3 A THREAT MODEL FOR AGENT AUTHORIZATION

We organize the space into six categories. Table 1 lists fifteen threats with the invariant that must hold, a one-line mitigation, and whether a single-token or server-manifest check can decide the threat before deployment. We then give, per category, the exploit condition and the defense.

Audience binding (MCP-01/02). *Exploit:* if a verifier accepts a token whose aud does not name it, or a client forwards its token to an upstream API, an attacker who obtains a token for one server replays or passes it through to another [? ?]. *Defense:* bind tokens to a resource (RFC 8707) and verify aud; the MCP specification prohibits passthrough. Both are decidable from the token and the server's declared identity.

Delegation (DEL-01.04). *Exploit:* without a verifiable act chain an agent's action cannot be attributed to the user who authorized it, and a sub-agent can exceed what was delegated [? ?]. *Defense:* carry and validate a nested act chain (static, DEL-01/02); enforce may_act on exchange and issue delegated rather than impersonated tokens (runtime, DEL-03/04) [? ? ?].

Scope (SCOPE-01.03). *Exploit:* an over-broad or non-expiring credential, or reuse of the human's credential, gives an injected agent durable, excessive authority [? ?]. *Defense:* least-privilege, expiring, agent-specific credentials. The scope and expiry a token carries are statically checkable (SCOPE-01/02); whether the agent reuses the human's credential is a runtime property (SCOPE-03).

Agent identity (AID-01/02). *Exploit:* treating a session as authentication, or giving the agent no distinct identity, makes revocation and attribution impossible [? ?]. *Defense:* a verifiable workload identity per agent (WIMSE, SPIFFE/SPIRE) [? ? ?]; runtime, because it needs a deployed identity substrate.

Confused deputy (CD-01/02). *Exploit:* an OAuth proxy that reuses consent across clients, or an abused dynamic registration, lets one party act with another's authority [? ?]. *Defense:* per-client consent, exact redirect_uri matching, and vetted registration, which are properties of a live consent flow and therefore runtime.

Agency (AGT-01/02). *Exploit:* injected input drives the agent to escalate privilege or call a poisoned tool [? ? ?]. *Defense:* per-action authorization or a human in the loop, and vetted tool descriptions [? ?]; runtime, because the decision depends on the executing context.

4 SYSTEMATIZING THE SPACE

Our systematization rests on three axes that, together, no prior work applies to agent authorization (§5).

(1) Pre-deployment testability. A threat is *statically decidable* when a defender can reproduce it from a single token, the issuer's key set, and the server's declared tool contract, *without running the agent*. By this criterion six of the fifteen threats are statically

decidable: audience binding (MCP-01/02), the act-chain delegation checks (DEL-01/02), and scope and expiry (SCOPE-01/02). Each is a property of claims a token carries or of a server's declared identity, so a defender can mint the adversarial token an attacker would and confirm the verifier rejects it, as a check that runs on every build. The remaining nine depend on a live consent flow (CD-01/02), server-side enforcement or a human-in-the-loop (DEL-03/04, SCOPE-03, AGT-01/02), or a deployed workload-identity substrate (AID-01/02), and cannot be settled from a token.

(2) Normative provenance. Each threat maps to the clause it violates: audience binding to RFC 8707 and the MCP passthrough prohibition; delegation to RFC 8693; scope and expiry to RFC 9700 and RFC 8725; registration to RFC 7591; confused deputy to CWE-441; agency to the OWASP LLM and Agentic Top-10s [? ?]. This lets a practitioner trace a defense to the authority that mandates it, a mapping the surveyed literature does not provide.

(3) The static/runtime enforcement line. The defenses in the literature scatter across a line they never name: *static* enforcement at the trust boundary (control/data-flow extraction [?], capability attestation, contract checks on tokens and key sets) versus *runtime* enforcement during execution (monotonic privilege confinement [?], pre-action authorization [?], mandatory access control [?], out-of-band policy [?]). Drawing the line explicitly re-frames several "runtime-only" defenses as compensating controls for threats a pre-deployment contract check could have excluded: if audience binding and scope are verified statically, a runtime controller has strictly less to contain.

An instrument for the static subset. We release an open-source analyzer that decodes a presented token and reports cited findings for the six statically decidable threats (audience binding, act-chain delegation, scope, and expiry) against a policy. It is stateless, makes no network calls, and runs in CI or beside the agent, so a team red-teams its own authorization before shipping. The taxonomy names the other nine as out of static scope, which is itself the result: it says what a pre-deployment tool, or a tightened standard, can and cannot be expected to guarantee.

5 RELATED WORK

We surveyed 44 research works on authorization, delegation, identity, and token security for agents and MCP (2023–2026); all arXiv identifiers were verified to exist. They divide into *threat taxonomies* and *point mechanisms*, and neither systematizes authorization by our three axes.

Taxonomies and surveys. MCP security surveys and SoKs organize by attacker type, lifecycle phase, defense placement, or ecosystem component [? ? ? ?]; protocol-level analyses dissect the MCP specification and its trust assumptions [? ?]. These name confused-deputy and passthrough as risks but do not sort threats by testability or bind them to normative clauses.

Point mechanisms. Delegation protocols bind actions to user intent and bound sub-agent authority [? ? ? ?]; privilege and access-control frameworks enforce least privilege and confinement at runtime [? ? ? ?]; identity work applies WIMSE/SPIFFE to agents [?

Table 1: Agent-authorization threats by category, with the invariant that must hold, a mitigation, the normative source, and whether the threat is *statically* decidable from a single token or server manifest before deployment (✓) or is irreducibly run-time (×). Sev.: severity.

ID	Sev.	Broken invariant (exploit)	Mitigation	Source	Static
<i>Audience binding</i>					
MCP-01	crit	token accepted with wrong/any audience (passthrough, replay)	verify aud names this resource; reject forwarded tokens	RFC 8707/9728; MCP [? ? ?]	✓
MCP-02	high	token carries no resource indicator (unbound audience)	request resource-bound tokens; require aud	RFC 8707 [?]	✓
<i>Delegation</i>					
DEL-01	high	missing or forged act claim (no accountable actor)	require a signed act claim on OBO tokens	RFC 8693 [?]	✓
DEL-02	high	unverifiable nested delegation chain (transitive-trust abuse)	validate the full nested act chain	RFC 8693 [?]	✓
DEL-03	high	may_act not enforced on exchange	enforce may_act at the token endpoint	RFC 8693 [?]	×
DEL-04	med	impersonation where delegation was required	issue delegated (not impersonated) tokens	RFC 8693 [?]	×
<i>Scope</i>					
SCOPE-01	high	over-scoped agent credential (admin:*)	least-privilege scopes per task	RFC 9700; OWASP [? ?]	✓
SCOPE-02	high	non-expiring / long-lived token	short exp; rotation	RFC 8725 [?]	✓
SCOPE-03	high	agent reuses the human's root credential	distinct agent credential	OWASP [?]	×
<i>Agent identity</i>					
AID-01	high	session used as authentication	bind to a verifiable workload identity	WIMSE/SPIFFE [? ?]	×
AID-02	med	no distinct workload/agent identity	issue per-agent SVID/credential	WIMSE/SPIFFE [? ?]	×
<i>Confused deputy</i>					
CD-01	crit	proxy acts on a client's behalf without per-client consent	per-client consent; exact redirect_uri match	CWE-441; MCP [? ?]	×
CD-02	high	dynamic client registration abuse	vet/limit DCR; bind registrations	RFC 7591 [?]	×
<i>Agency</i>					
AGT-01	crit	prompt-injection-driven privilege escalation	per-action authorization; human-in-loop	OWASP [? ?]	×
AGT-02	high	tool poisoning / rug-pull via tool metadata	vet tool descriptions; pin versions	OWASP [?]	×

[? ?]; and by-design and detection defenses target prompt injection and tool poisoning [? ? ? ?]. Each is a single point on the static/runtime line; none draws the line.

The coverage gap. Scoring every work on seven threat categories (Table 2) makes the gap visible: two columns are thin. Audience binding is a central contribution in essentially one research work (Agentic JWT [?]) plus one enterprise framework [?], and confused-deputy / registration appears only as a named taxonomy entry, almost never as a verified or enforced property. Prompt injection and tool poisoning, by contrast, are densely covered. The field has thoroughly characterized the model-level attacks and under-formalized the token- and consent-level authorization guarantees, the seam this SoK occupies.

6 DISCUSSION: WHAT A STANDARD COULD MANDATE

The testability axis has a direct standardization reading. The six statically decidable threats are exactly the ones a profile could make mandatory-to-verify and a conformance test could check: an MCP or OAuth profile that *requires* resource indicators (RFC 8707), rejects passthrough, mandates nested act validation, and caps scope and lifetime would move these threats from “deployer’s responsibility, usually missed” to “checked at the boundary.” The nine runtime threats cannot be closed by a static profile and instead mark where the ecosystem still needs deployed consent, delegation enforcement, and workload identity.

7 THREATS TO VALIDITY

The MCP specification is fast-moving; this systematization is a dated snapshot. The static/runtime line depends on token and manifest availability; a deployment that hides relevant state reduces what any static check can conclude. Our survey covers research indexed by 2026-10; most agent-authorization work is recent and not yet peer-reviewed, which we mark. A prevalence measurement of the statically checkable weaknesses across public MCP deployments is future work, with a hand-validated sample and coordinated disclosure.

ETHICS CONSIDERATIONS

The systematization uses only public specifications, advisories, and CVEs. The accompanying analyzer runs on tokens the operator already holds and sends nothing anywhere; we probe no third-party agents or servers and use no real user tokens. Any future measurement will characterize population prevalence, not single deployments, and will follow coordinated disclosure for identifiable high-severity findings. No personal data or human subjects are involved.

OPEN SCIENCE

We release the static analyzer and a machine-readable threat taxonomy (each threat with its invariant and normative sources) under an open licence, so the systematization is inspectable and the statically checkable subset is reproducible. A DOI-pinned artifact will accompany the camera-ready version.

Table 2: Coverage of the seven agent-authorization threat categories across representative works. $\checkmark$: a central contribution; $\circ$: partially addressed; blank: not addressed. AudBind, audience binding and passthrough; OBO, on-behalf-of delegation; Scope, scope minimization; ConfDep, confused deputy and dynamic registration; WID, workload/agent identity; PI, prompt-injection escalation; Tool, tool poisoning. Two columns, AudBind and ConfDep, are thin.

Work	AudBind	OBO	Scope	ConfDep	WID	PI	Tool
<i>Taxonomies and surveys</i>							
MCP Landscape [?]	$\circ$	$\circ$	$\circ$	$\circ$	$\circ$	$\checkmark$	$\checkmark$
SoK MCP [?]	$\circ$	$\circ$	$\circ$	$\circ$	$\circ$	$\checkmark$	$\checkmark$
MCP-38 [?]	$\circ$	$\circ$	$\circ$	$\checkmark$	$\circ$	$\checkmark$	$\checkmark$
Enterprise MCP [?]	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\circ$	$\checkmark$	$\checkmark$
Breaking the Protocol [?]	$\circ$			$\circ$	$\circ$	$\checkmark$	$\checkmark$
<i>Delegation and identity mechanisms</i>							
Agentic JWT [?]	$\checkmark$	$\checkmark$	$\checkmark$	$\circ$	$\checkmark$	$\circ$	
AIP [?]	$\circ$	$\checkmark$	$\circ$	$\circ$	$\checkmark$		
Bounded Agents [?]	$\circ$	$\checkmark$	$\checkmark$	$\circ$	$\circ$	$\circ$	
AID-Guard [?]	$\circ$	$\checkmark$	$\checkmark$	$\circ$	$\circ$	$\circ$	
SAGA [?]	$\circ$	$\checkmark$	$\checkmark$	$\circ$	$\checkmark$	$\circ$	
AI Identity [?]	$\circ$	$\checkmark$	$\circ$	$\circ$	$\checkmark$		
aiAuthZ [?]	$\checkmark$	$\circ$	$\circ$	$\circ$	$\checkmark$		
<i>Privilege / runtime control</i>							
Progent [?]			$\checkmark$			$\checkmark$	$\circ$
Pre-Action Authz [?]	$\circ$	$\circ$	$\checkmark$	$\circ$	$\circ$	$\checkmark$	
MAC framework [?]		$\circ$	$\checkmark$	$\circ$		$\checkmark$	$\circ$
IsolateGPT [?]			$\checkmark$	$\circ$	$\circ$	$\checkmark$	$\circ$
If Agents Were Angels [?]	$\circ$	$\circ$	$\checkmark$	$\circ$	$\circ$	$\checkmark$	$\circ$
<i>Model-level attacks and defenses</i>							
Indirect PI [?]						$\checkmark$	$\circ$
CaMeL [?]			$\checkmark$	$\circ$		$\checkmark$	$\circ$
MCPTox [?]			$\circ$			$\checkmark$	$\checkmark$
No-Box MCP [?]						$\checkmark$	$\checkmark$

8 CONCLUSION

Agent authorization is a high-stakes surface the research literature has studied around rather than through. We systematize it by pre-deployment testability, by normative provenance, and by the static-versus-runtime enforcement line; we show that six of fifteen threats are contract-verifiable before an agent runs and nine are irreducibly runtime; and we argue this line should guide what a pre-deployment tool checks and what a standard makes mandatory to verify.

LLM USAGE STATEMENT

LLMs were used for editorial purposes in this manuscript, and all outputs were inspected by the authors to ensure accuracy and originality. An LLM-assisted process additionally supported the literature survey of §5: candidate works were gathered with model assistance and each was then verified by the authors to exist, with titles and author lists confirmed against the arXiv API on 2026-10-06; works that could not be verified were excluded, and one paper that its own authors had formally withdrawn was identified this way and removed. The authors are responsible for the selection of works, the coverage judgments in the systematization, and every claim in the paper. One reproducibility limitation applies: the assisting models are not openly available, so the survey's search recall cannot be reproduced exactly, although every cited work is independently verifiable by its arXiv identifier.